

CLIPMIND AI

AI-Powered Video Summarization and Intelligent Key Moment Detection Platform

Milestone 3 Documentation

Additional Features: Functionality

1. Introduction

ClipMind AI is an AI-powered video learning platform that helps users understand and revise video content. Milestone 3 focuses on four additional learning features: Semantic Q&A , Multilingual Translation, AI Study Notes, Self Assessment through Learning Questions.

2. Objectives of the Additional Features

- Help users find relevant information in video transcripts using semantic search.
- Make video content accessible in supported languages.
- Help learners revise video content through automatically generated study notes.
- Support self-review with generated learning questions and hints.

3. Feature 1 — Semantic Q&A

Semantic transcript search using embeddings and cosine similarity

3.1 Purpose

Semantic Q&A helps users find information related to a question or search phrase within a video's transcript. Long videos can contain a large amount of information, making it time-consuming to locate one particular concept manually.

This feature uses semantic similarity to retrieve relevant transcript passages. Rather than depending only on exact keyword matches, it represents text as numerical vectors called embeddings and compares their similarity.

Important distinction: The inspected backend implements semantic retrieval of relevant transcript chunks. This is not, by itself, proof of a separate AI-generated conversational answer. The documentation should describe the functionality that the current interface and backend actually provide.

3.2 Functionality

The feature supports the following processing:

Transcript chunking: The transcript is divided into smaller, overlapping text chunks.

Embedding generation: The embedding model converts the text chunks into numerical vector representations.

Query processing: The user's query is encoded using the same embedding model.

Semantic similarity: Cosine similarity is used to compare the query vector with transcript-chunk vectors.

Result ranking: Relevant chunks are returned according to the search logic and requested result limit.

Video-specific search: The endpoint searches the transcript associated with the selected video and receives the authenticated user's context.

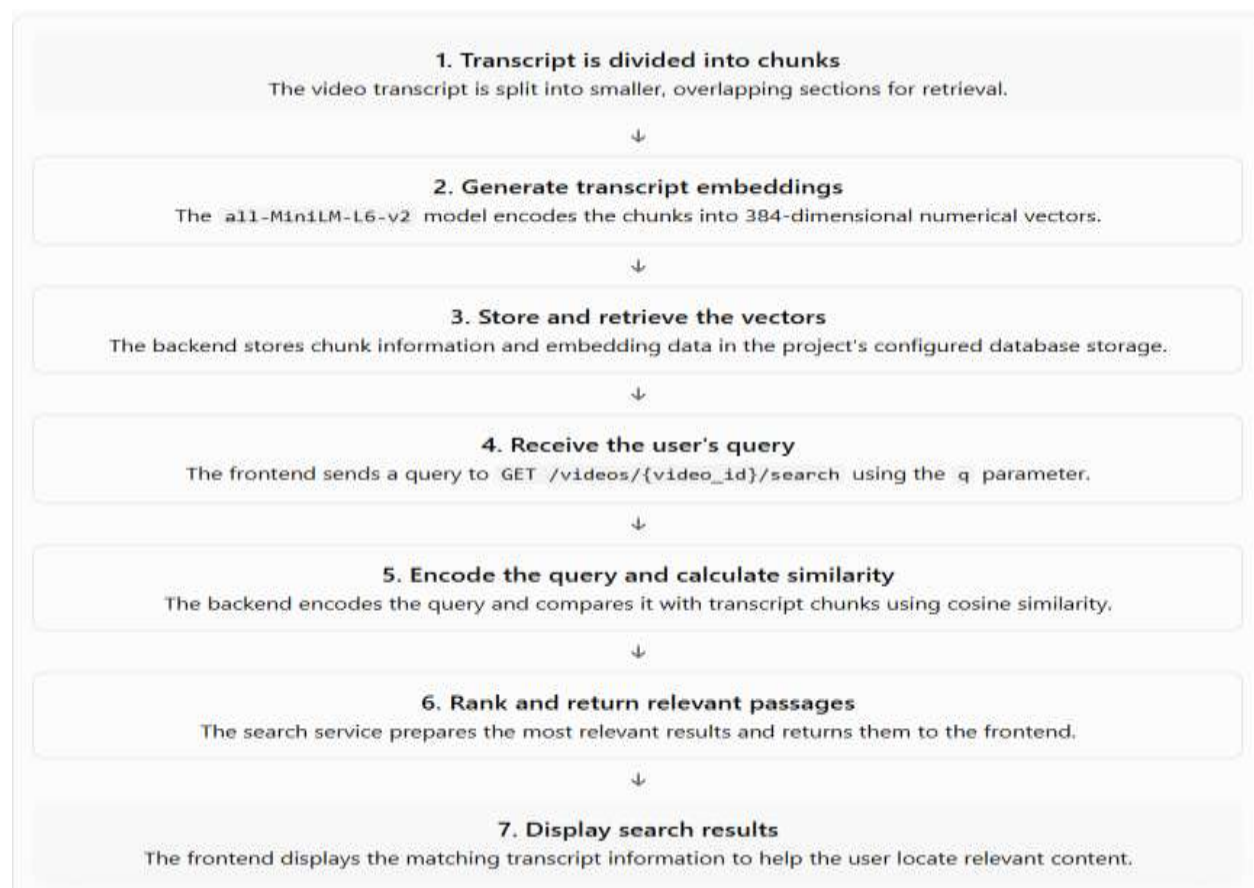
Result display: The frontend can display the retrieved transcript passages so users can locate the relevant information.

3.3 Technologies and Components Used

Component	Purpose
Python	Implements embedding and semantic-search processing.
FastAPI	Exposes the transcript-search endpoint.

Sentence Transformers	Provides the text-embedding model.
all-MiniLM-L6-v2	Encodes queries and transcript chunks into 384-dimensional vectors.
Cosine similarity	Measures similarity between the query and transcript-chunk vectors.
PostgreSQL	Stores video, transcript and related application data.
SQLAlchemy	Provides database access in the backend.
Vector-compatible storage / pgvector	Supports embedding storage where configured in the project.
search_transcript_similarity()	Performs the backend's transcript similarity-search operation.
Authentication dependency	Supplies the current user to the protected endpoint.
Next.js and React	Provide the query interface and display the search results.

3.4 Backend Workflow



3.5 Detailed Backend Processing

Step 1 — Transcript chunking: The transcript is split into smaller, overlapping chunks. Chunking allows the system to compare the user's query with individual portions of the transcript rather than treating the entire transcript as one large text block.

Step 2 — Embedding generation: The all-MiniLM-L6-v2 model from Sentence Transformers converts each chunk into a 384-dimensional vector. These vectors represent semantic information about the text and support similarity comparisons.

Step 3 — Embedding storage: The embedding service stores chunk and vector information in the project's database layer. The available project code references PostgreSQL and vector-compatible storage, including pgvector-related support. The exact storage mechanism should be described according to the active configuration.

Step 4 — Query submission: The backend provides the endpoint:

GET /videos/{video_id}/search

The important parameters are:

video_id: identifies the video to search.

q: the required, non-empty search query.

top_k: the maximum number of results requested; the route uses a default value of 5.

Step 5 — Semantic comparison: The backend calls `search_transcript_similarity()` with the query, video ID, authenticated user ID, database session and result limit. The search process encodes the query and compares it with transcript-chunk embeddings using cosine similarity.

Step 6 — Result preparation: The service returns matching transcript information in ranked results. Depending on the active implementation, result objects can include chunk text, similarity information, chunk index or an importance score.

Step 7 — Missing embeddings: If no embeddings are available for the video, the route can return an empty results list and a message indicating that the summary or embedding-generation process needs to run first.

3.6 How the Feature Works

Semantic Q&A connects a user's query with the relevant sections of a video transcript. The embedding model represents the query and transcript chunks as vectors, and cosine similarity helps identify the closest semantic matches. The backend returns the matching passages for the frontend to display.

This reduces the effort required to search through a long transcript manually. The confirmed implementation is a semantic retrieval system; do not claim that it generates natural-language answers unless a separate answer-generation component is implemented and tested.

SEMANTIC QA

Ask a question about this video

what is normal algorithm

Ask Question

Top Answer found

"where a normal algorithm could be a set of instructions, such as how to turn a pile of raw ingredients into your favorite honeymusted chicken dish. The set of instructions might start out by saying, first, cut up the vegetables, then season the chicken, then preheat the oven, etc. And if you follow these steps correctly, you'll end up with your favorite honeymusted chicken dish. Oh, that's making me hungry, actually. What's important to note here is you started with an input. You're set of ingredients, and a set of instructions on what to do to get to your favorite dish. What happens with the machine learning algorithm is instead of starting with an input and a set of instructions, you start with an input and ideal output. In our case, the ingredients is the input, and the output is our favorite chicken dish."

Jump to 1:05

Similarity Match: 47.4%

Figure 1. Semantic Q&A

4. Feature 2 — Multilingual Translation

4.1 Purpose

Multilingual Translation is an additional feature of ClipMind AI designed to make video-based learning more accessible to users who prefer different languages. Educational videos may contain technical terms and explanations that are difficult for some learners to understand in the original language. This feature aims to help users understand video-related textual content in a supported target language.

The feature is intended to reduce language barriers and allow learners to engage with the information extracted from a video more comfortably.

4.2 Functionality

The feature allows the application to work with video-related text and a target language. Depending on the functionality enabled in the current application build, the translated content may include a transcript, summary, or other supported text associated with the video.

Its main functions are:

- **Language selection:** The user chooses a supported target language when the language-selection interface is available.
- **Text translation:** The source text is processed by the translation component to produce content in the selected language.
- **Translated content display:** The frontend displays the translated result in the relevant section of the video page.
- **Improved accessibility:** Users can understand video information in a language they are more comfortable with.

Translation is performed on text associated with the video. It is separate from the original video transcription process, which extracts spoken content from the video.

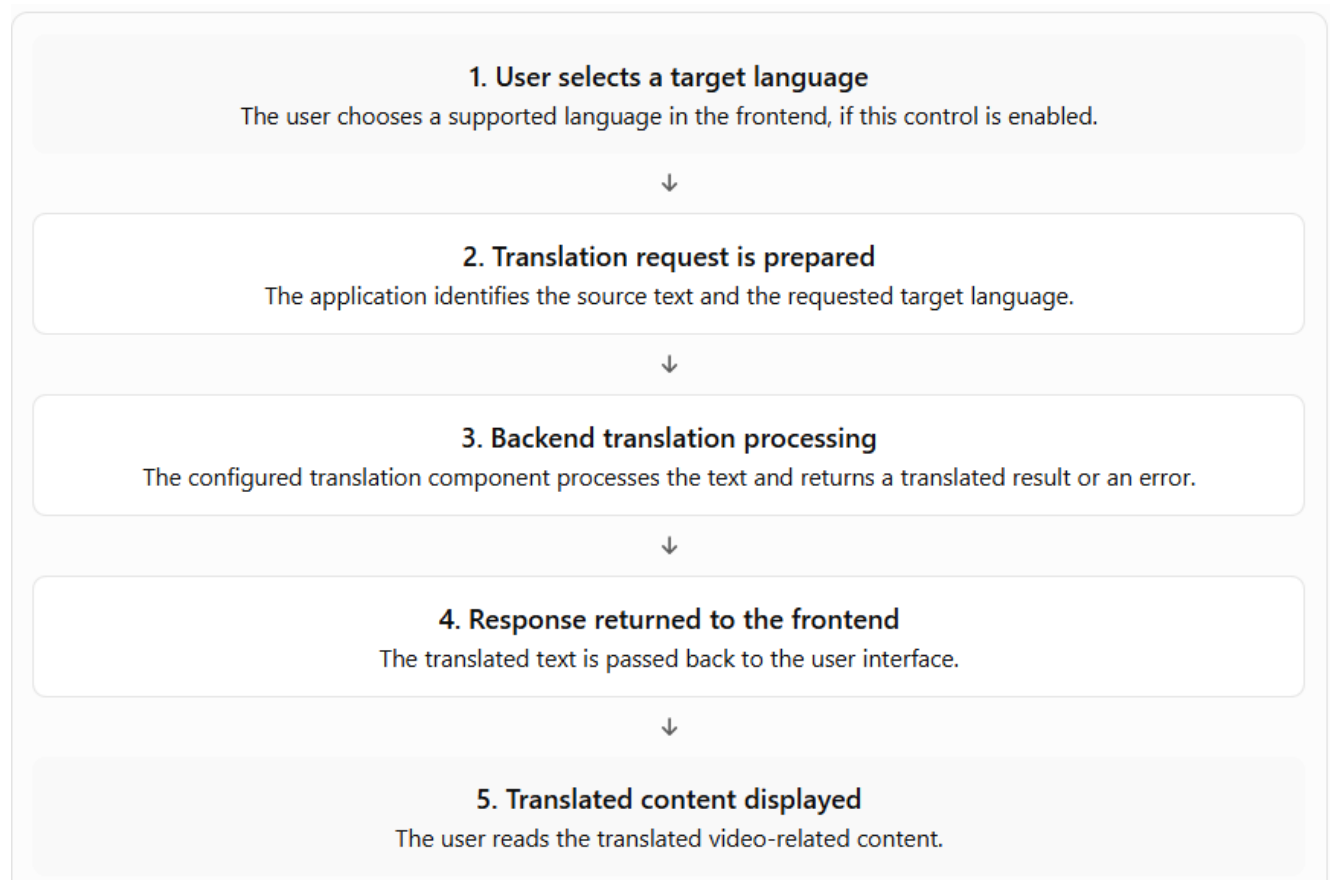
4.3 Technologies and Components Used

Component	Purpose
Next.js and React	Provide the frontend interface and display translated content.
FastAPI backend	Provides the backend API framework used by ClipMind AI. The exact translation route should be verified in the current code.
Translation component or service	Converts source text into the selected target language. The exact provider or library needs to be confirmed from the implementation.

Source text and target-language input	Supply the content to translate and the requested output language.
API request and response handling	Transfers the translation request and returns the translated result to the frontend.

Implementation note: The exact translation provider, Python package, and API endpoint have not been conclusively verified from the available project information. Do not name a particular translation library or claim that a specific model is used unless it is present in your current source code.

4.4 Backend Workflow



4.5 How the Feature Works

The frontend collects the language selection and identifies the text that needs translation. The request is then passed through the translation flow implemented in the application. The translation component processes the supplied text and returns the result, which the frontend displays to the user.

The quality of the translated output depends on the source text, the supported languages, and the translation service or model configured in the application.

MULTILINGUAL

Offline Translation (Meta NLLB-200)

Completed

Hindi (हिन्दी)

Translate Content

Translated Summary

और एक मशीन लर्निंग एल्गोरिथ्म क्या है यह इनपुट, कच्ची सामग्री को देखता है, और फिर यह उत्पादन, पसंदीदा चिकन पकवान को देखता है, और यह इन दोनों के बीच निर्देशों के सेट को समझने की कोशिश करता है। To make a way. एक वाक्य में, मशीन लर्निंग एक एल्गोरिथ्म, या कंप्यूटर प्रोग्राम का उपयोग डेटा में विभिन्न पैटर्न के बारे में जानने के लिए किया जाता है, और फिर उस एल्गोरिथ्म को लिखा जाता है और इसे भविष्य के बारे में भविष्यवाणी करने के लिए इसी तरह के डेटा का उपयोग करते हुए सीखा जाता है। चलो एक उदाहरण का उपयोग करते हैं, जहां एक सामान्य एल्गोरिथ्म निर्देश का एक सेट हो सकता है, जैसे कि कैसे अपने पसंदीदा मधुमक्खियों वाले चिकन पकवान में कच्चे अवयवों के ढेर को बदलने के लिए, और क्या एक मशीन लर्निंग एल्गोरिथ्म है यह इनपुट, कच्ची सामग्री को देखता है, और फिर यह उत्पादन, पसंदीदा चिकन पकवान को देखता है, और यह इन दोनों के बीच निर्देशों के सेट को समझने की कोशिश करता है। यदि आप सामग्री और आदर्श आउटपुट के सेट को देखते हैं, तो आपका पसंदीदा चिकन पकवान, सौ से अधिक बार, आपको शायद बहुत अच्छा लगता है, या यह जानने के लिए बहुत करीब है कि किस निर्देश का सेट उस डिश को बनाना है। हमारे चिकन पकवान उदाहरण में, एक मशीन लर्निंग एल्गोरिथ्म को सही सामग्री दिए गए स्वादिष्ट चिकन पकवान बनाने का एक तरीका मिल सकता है।

Translated Transcript

चलो मशीन लर्निंग के साथ शीर्ष से शुरू करते हैं। जबकि मशीन लर्निंग व्यापक है, इसमें कई अलग-अलग पहलू शामिल हैं, और आप इसे ऑनलाइन पर कई अलग-अलग परिभाषाएं देखेंगे, लेकिन इस कोर्स के लिए, हम इसे व्यावहारिक रखने जा रहे हैं। एक वाक्य में, मशीन लर्निंग एक एल्गोरिथ्म, या कंप्यूटर प्रोग्राम का उपयोग डेटा में विभिन्न पैटर्न के बारे में जानने के लिए किया जाता है, और फिर उस एल्गोरिथ्म को लिखा जाता है और इसे भविष्य के बारे में भविष्य के समान डेटा का उपयोग करके भविष्यवाणी करना सीखा जाता है। मशीन लर्निंग एल्गोरिथ्म को मॉडल भी कहा जाता है, और हम पाठ्यक्रम में विनिमय शब्द का उपयोग करेंगे, कैसे मशीन लर्निंग एल्गोरिथ्म कंप्यूटर प्रोग्राम में सामान्य एल्गोरिथ्म से अलग है सीखने का पहलू, चलो एक उदाहरण का उपयोग करते हैं, जहां एक सामान्य एल्गोरिथ्म निर्देश का एक सेट हो सकता है, जैसे कि कैसे अपने पसंदीदा मधुमक्खियों वाले चिकन पकवान में कच्चे अवयवों के ढेर को बदलने के लिए, निर्देशों का सेट कहकर शुरू हो सकता है, पहले, सब्जियों को काट लें, फिर चिकन का मौसम करें, फिर ओवन को पहले से गरम करें, आदि। यदि आप इन चरणों को सही ढंग से पालन करते हैं, तो आप अपने पसंदीदा मधुमक्खियों के साथ समाप्त हो जाएंगे, ओह, यह मुझे भूखा बना रहा है, वास्तव में, यहां क्या ध्यान देना महत्वपूर्ण है? आप सामग्री का सेट कर रहे हैं, और आपके पसंदीदा पकवान को पाने के लिए क्या करना है, इस पर एक निर्देश, मशीन लर्निंग एल्गोरिथ्म के साथ क्या होता है, इसके बजाय इनपुट और निर्देशों का एक सेट शुरू होता है, आप इनपुट और आदर्श आउटपुट के साथ शुरू होते हैं। हमारे मामले

Figure2 . Multilingual Translation

5. Feature 3— AI Study Notes

AI-generated revision material from video content

5.1 Purpose

AI Study Notes is designed to help learners revise video content without repeatedly watching the entire video or reading a long transcript. It converts processed video information into structured, readable notes that are easier to review.

The feature helps organize the information contained in the video into a format suitable for learning and revision. Instead of manually writing notes while watching a video, users can review the notes generated by the application.

5.2 Functionality

The AI Study Notes feature uses processed video content as its source and generates notes in a readable format.

Its main functions are:

- Transcript-based generation: Uses the available video transcript as the primary source of information.
- Context-aware processing: Can use visual-context information when it is available and supplied to the notes service.
- AI-powered note generation: Sends the prepared source information to the project's AI text-generation function.
- Structured output: Produces Markdown-formatted notes so that headings and sections can be displayed clearly.
- Revision support: Allows users to review important video information from the video page.

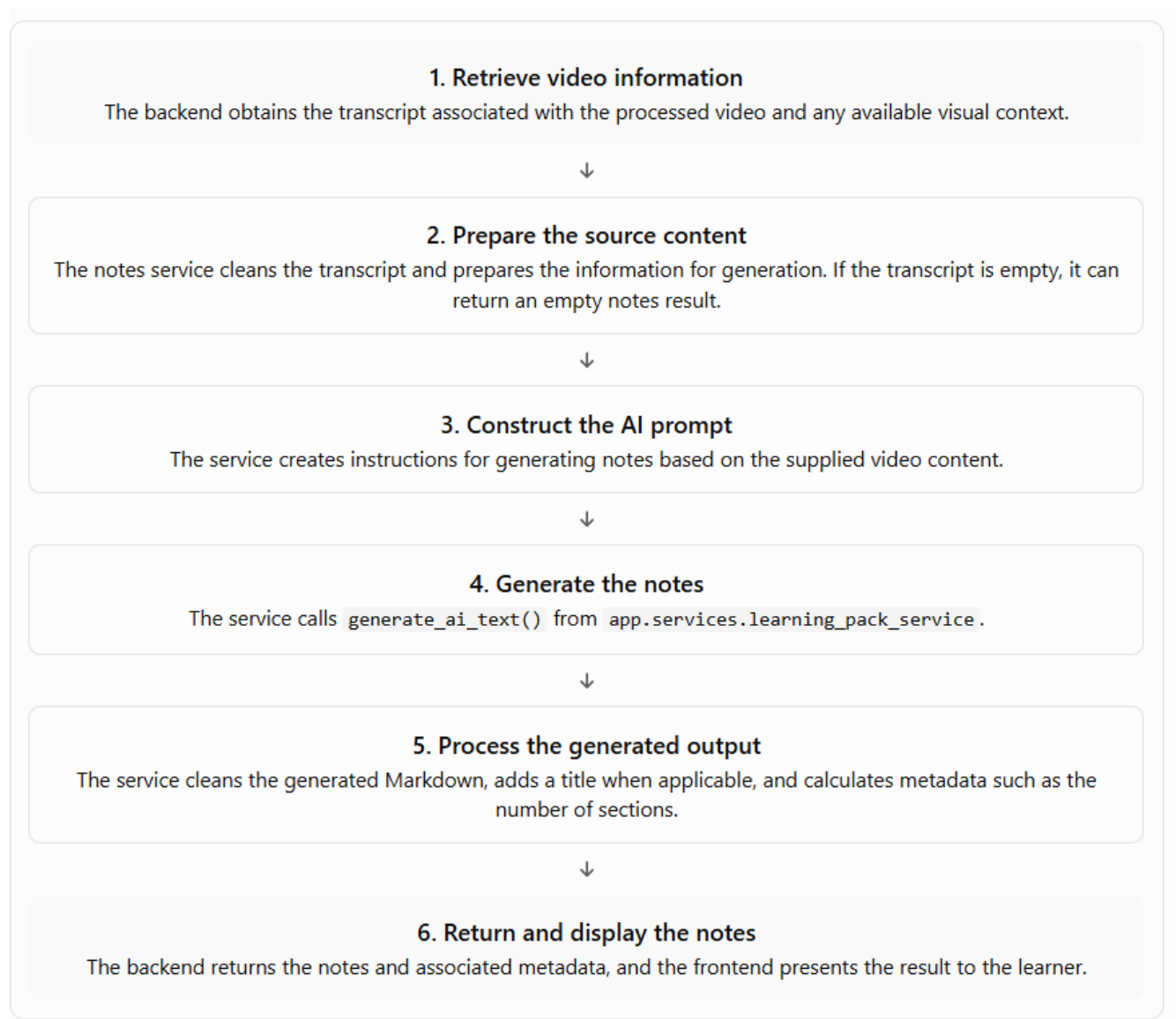
The notes are generated from the information provided to the AI component. Their quality therefore depends on transcript quality, available context, and the generated response.

5.3 Technologies and Components Used

Component	Purpose
Python	Implements the notes-generation and text-processing logic.
FastAPI	Provides the backend API framework.

<code>app.services.learning_pack_service</code>	Contains the AI text-generation function used by the notes service.
<code>generate_ai_text()</code>	Generates notes from the prepared prompt and source information.
Video transcript	Supplies the primary text for generating notes.
Visual context	Provides additional video-related context when available.
Markdown	Structures the generated notes into readable sections.
Next.js and React	Display the generated notes in the frontend.

5.4 Backend Workflow



5.5 Detailed Backend Processing

Transcript preparation: The transcript acts as the primary source for note generation. The service cleans the text before using it in the prompt. When visual-context items are available, they can be included to provide additional information.

AI prompt generation: The service constructs instructions for the AI text-generation component. This step guides the generation process so that the notes remain related to the supplied video content.

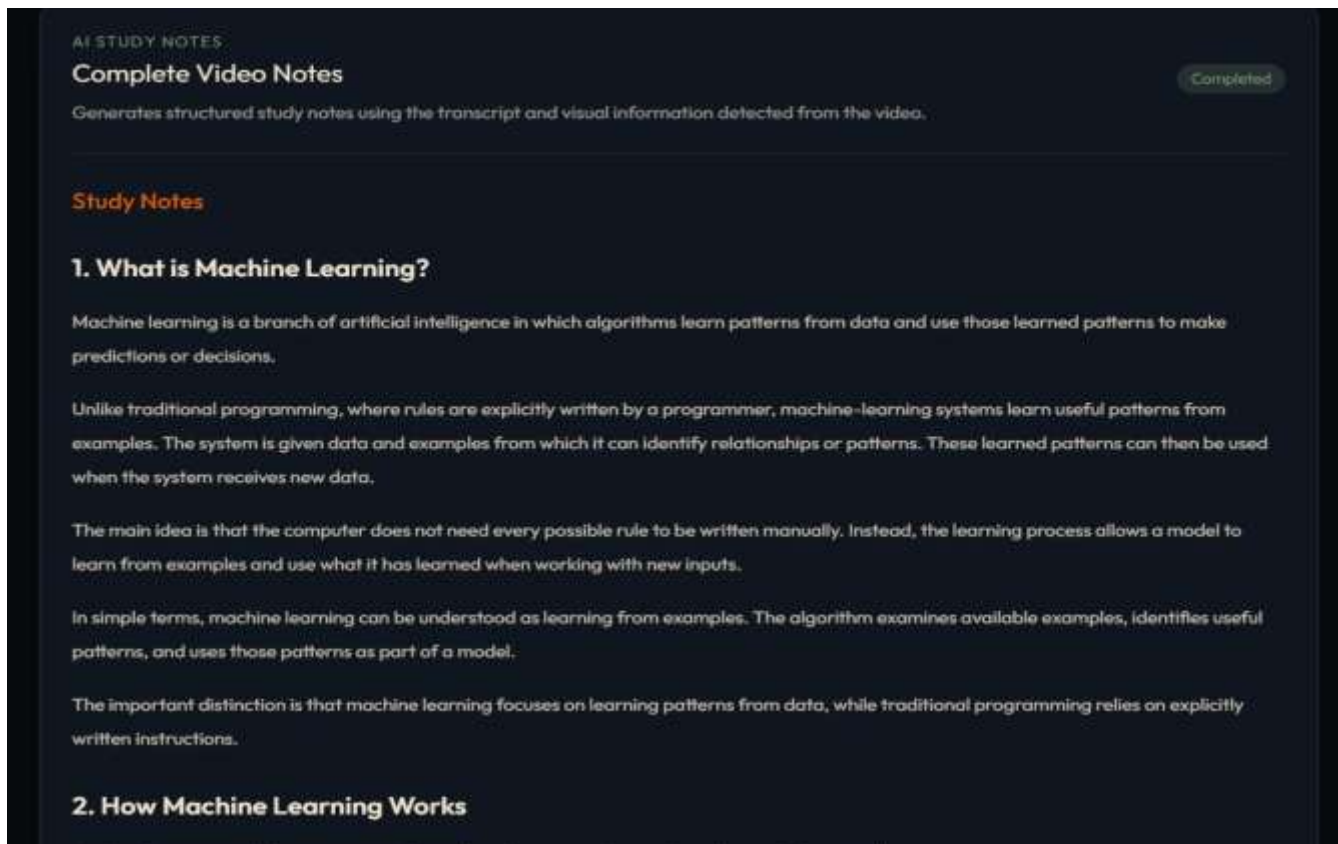
AI text generation: The function `generate_ai_text()` is called to generate the notes. The exact underlying language model or external provider should only be named if confirmed in the implementation of that function.

Output formatting: The returned text is cleaned and prepared as Markdown. Metadata such as section count and the number of visual-context items used may also be returned.

Error handling: If generation fails, the service can return an empty notes result with zero sections rather than presenting a successful result that was never generated.

5.6 How the Feature Works

The feature connects the processed video transcript with an AI text-generation component. The backend prepares the text and context, generates the notes, formats the result, and returns it to the frontend. The frontend then displays the structured notes so the learner can review the video's key information.



4. Machine Learning vs Traditional Programming

Traditional programming and machine learning both use data and produce outputs, but they differ in how the rules or patterns used to produce those outputs are obtained.

Traditional Programming	Machine Learning
Programmer explicitly defines the rules	Model learns patterns from examples
Rules and input are provided to produce an output	Examples of inputs and outputs are used to learn patterns
The program follows predefined instructions	The learned model applies patterns discovered from data
To change the behavior, the programmed rules generally need to be changed	The behavior depends on the patterns learned during the learning process

In traditional programming, the programmer decides the rules and writes them explicitly. The computer then follows those instructions when processing the input.

In machine learning, the programmer provides examples or data and the algorithm learns useful patterns from them. The resulting model can then apply those learned patterns to new data.

The main difference is therefore the source of the rules: traditional programming uses explicitly written rules, while machine learning learns patterns from examples.

5. Data Mining, Data Science, Machine Learning

Figure 3. AI Study Notes

6. Feature 4 — Self Assessment / Learning Questions

Question generation, hints and timestamp-based review

6.1 Purpose

Self Assessment — Learning Questions is designed to help learners review and reflect on the concepts explained in a video. It presents questions related to the processed video content so that learners can revisit the material and check their understanding.

Instead of requiring the learner to search manually through the entire video, the feature connects learning questions with the relevant point in the video.

6.2 Functionality

The confirmed implementation provides the following functionality:

- Learning-question generation: Questions are generated from video-analysis information, including relevant key-moment or transcript chunks.
- Question display: The learner can view the generated questions on the video page.
- Hints: The interface displays hints associated with the questions.
- Timestamp-based review: The “Review at” action takes the learner to the corresponding point in the video.
- Learner-oriented interface: The inspected frontend displays this section to users with the learner role.

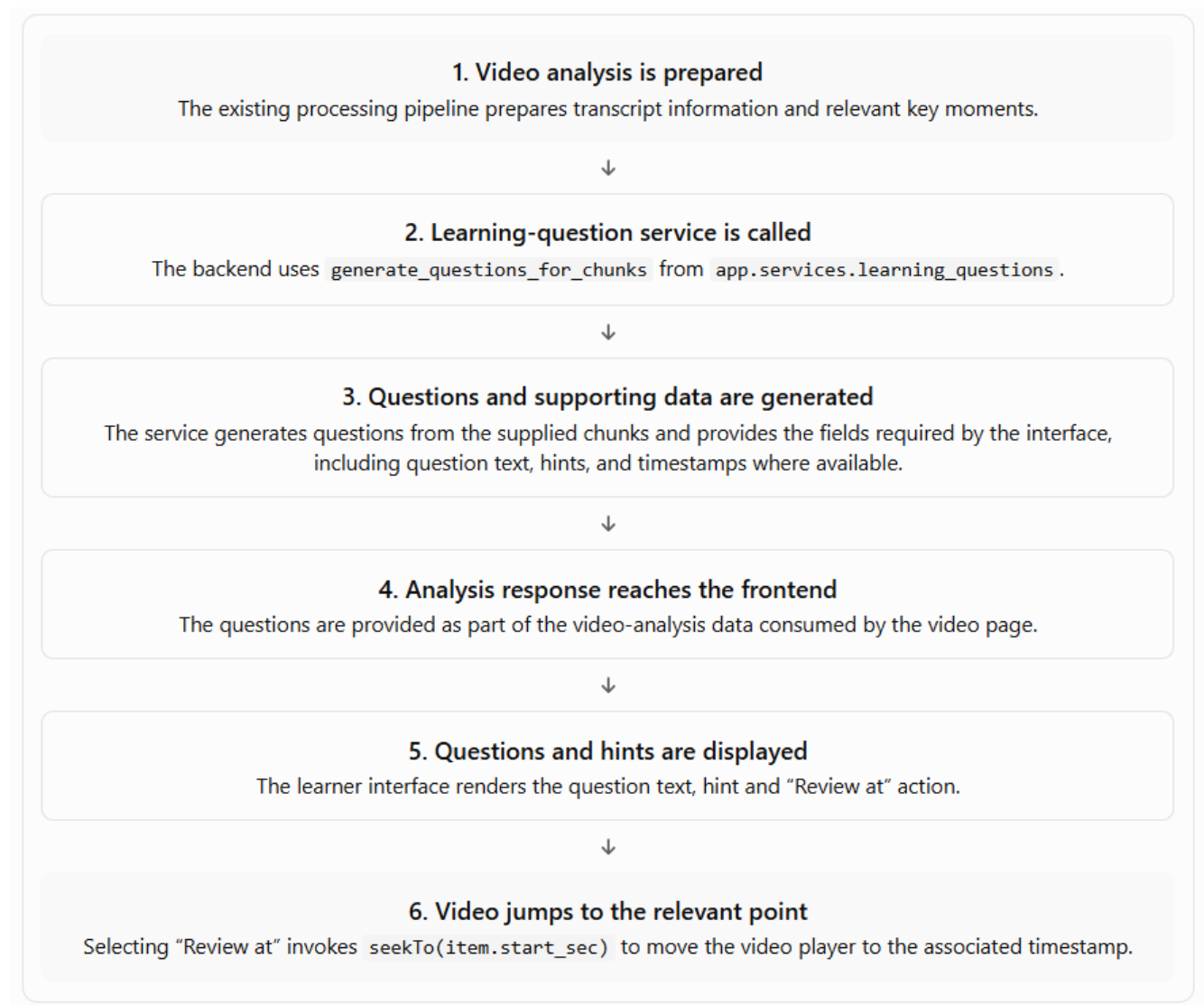
The inspected implementation confirms question and hint display and timestamp navigation. It does not confirm answer submission, automatic grading, or a final assessment score. These capabilities should not be described as implemented unless they are present and tested in the current version.

6.3 Technologies and Components Used

Component	Purpose
Python	Implements the backend learning-question generation logic.
FastAPI	Connects video-analysis processing with the frontend through backend routes.
app.services.learning_questions	Provides the learning-question generation component.
generate_questions_for_chunks	Generates learning questions from the supplied chunks.

Transcript and key-moment chunks	Provide source material for the generated questions.
question and hint data	Supply the question text and supporting hint displayed in the interface.
start_sec	Identifies the timestamp associated with a question.
Next.js and React	Render the learning-question interface and handle user interactions.
seekTo()	Moves the video player to the relevant timestamp when the learner selects “Review at.”

6.4 Backend Workflow



6.5 Detailed Backend and Frontend Processing

Question generation: The backend uses `generate_questions_for_chunks` to generate questions from the video-analysis chunks. These chunks connect the questions to the material covered in the video.

Data transfer: The generated question data is included in the video-analysis response. The frontend uses the returned information to populate the Learning Questions section.

Question and hint display: The React component renders the available question text and its hint. The learner can read the question and use the hint to guide further review.

Timestamp navigation: The question data includes a starting timestamp when available. When the learner selects “Review at,” the frontend invokes `seekTo(item.start_sec)`. This allows the learner to revisit the source explanation associated with the question.

Scope of assessment: In the inspected implementation, the feature supports question-based review. The available code does not establish a complete quiz-submission pipeline or automated evaluation of user answers.

6.6 How the Feature Works

The backend generates learning questions from video-related content and supplies them to the frontend. The learner can read each question, view its hint, and navigate to the corresponding video timestamp to review the explanation. This connects the assessment experience with the original learning material.

SELF ASSESSMENT

5

Learning Questions

Q- What idea does the example about "features making" illustrate?

Hint: Look past the details of the example and identify what "features making" is meant to show.

Review at 4:49

Q- What changes as the process involving "where normal" moves forward?

Hint: Track the input, action, and result described for "where normal".

Review at 1:05

Q- What central claim does the video make about "instructions you", and how is it supported?

Hint: Identify the claim about "instructions you", then recall the explanation or evidence that follows it.

Review at 2:12

Q- What makes "let's start" different from related ideas?

Hint: Compare the description of "let's start" with the contrast made in the video.

Review at 0:00

Q- What changes as the process involving "dish hundred" moves forward?

Hint: Track the input, action, and result described for "dish hundred".

Review at 3:32

Figure 4. Semantic Q&A